Claims Orchestration on Temporal

Approach B: Claim Entity Workflow · draft for review · implementation starts once an approach is chosen.

1. Summary

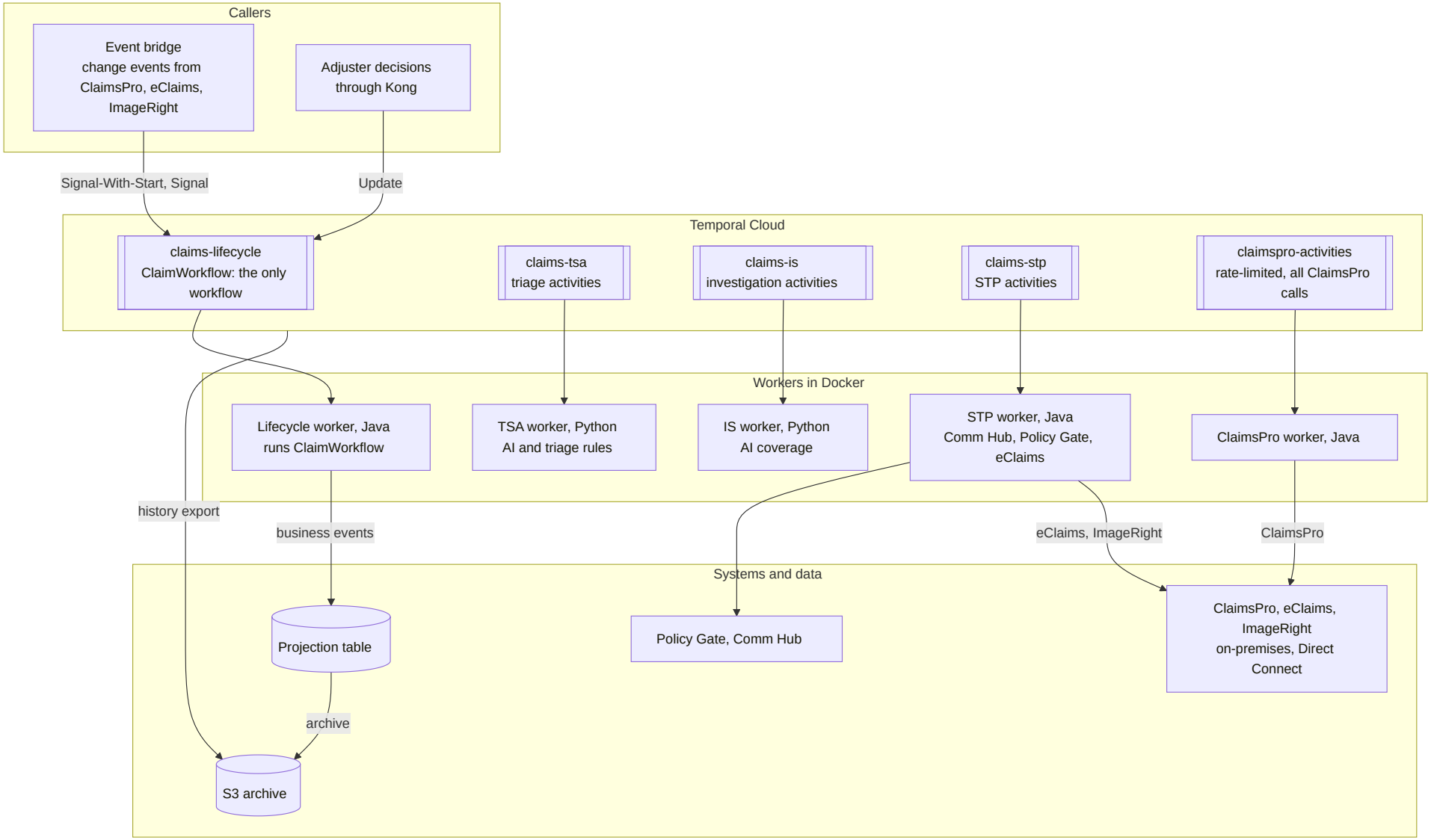
Each claim is one long-running **ClaimWorkflow**: the standard Temporal entity-workflow pattern, one workflow per business entity for its whole life. Triage, investigation and STP are modules inside it, not separate workflows. Their work runs as activities on each track's own task queue and worker deployment: the Python team's AI and triage-rule activities on claims-tsa and claims-is, the Java STP activities on claims-stp, every ClaimsPro call on the rate-limited claimspro-activities queue. Events and every human decision arrive at the same workflow ID, `claim:{claimId}`. Payment runs in a region of the workflow that is never cancelled. Application data, S3, contracts and security are the same production baseline as Approach A.

2. Why one workflow

The client asked whether a child workflow is needed at each stage. Temporal's guidance is that child workflows are not a way to organise code; they are justified by a separate service or worker fleet that owns the workflow, a need to partition a very large history, or a one-to-one mapping to a separate resource. In this claims platform the tracks need separate *workers* (Python AI, Java STP) and separate *deployments*, which activities on their own task queues already give. The orchestration itself (stage, events, decisions, re-triage) is one coherent state machine for one claim, so it lives in one workflow.

Property	What one workflow gives
One history per claim	Everything that happened to a claim is in one execution (and its Continue-As-New chain)
One front door	Events, decisions and queries all go to `claim:{claimId}`; nothing is forwarded or relayed
One place for cross-track rules	Re-triage limits, material change, stage version, payment authority live together
Fewer actions	No child workflow starts (two actions each) and no forwarded signals
Independent deployment where it matters	Activity workers (Python AI, Java STP) deploy without workflow versioning concerns, as long as the activity contract holds

3. Temporal architecture

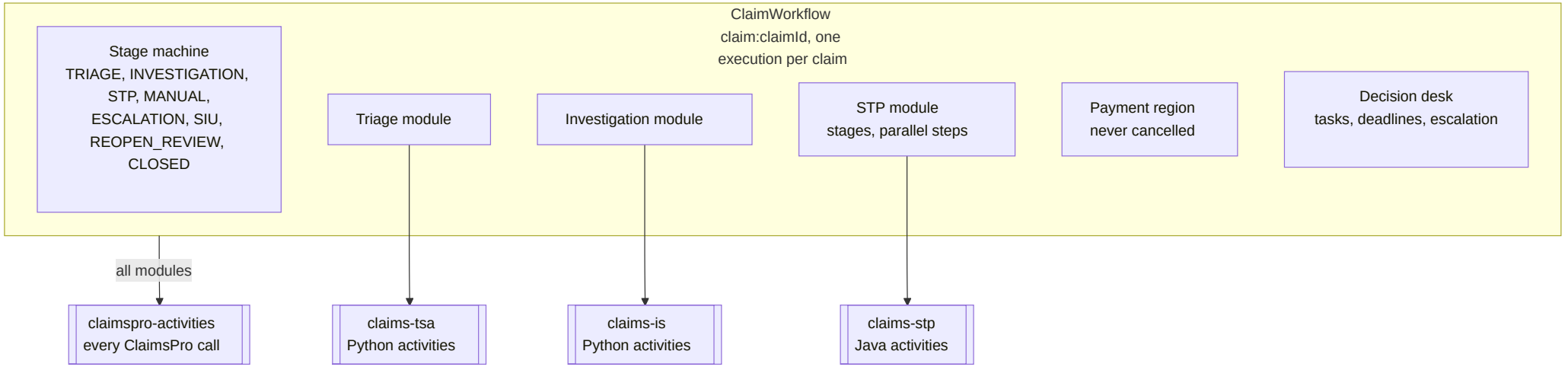


Task queue	Runs	Worker deployment
claims-lifecycle	ClaimWorkflow (the only claim workflow), ArchiveWorkflow; projection, policy and archive activities	Lifecycle worker, Java (Auto-Upgrade)

Task queue	Runs	Worker deployment
claims-tsa	Ai.assessRisk, Triage.applyRules	TSA worker, Python (activities only)
claims-is	Ai.analyzeCoverage	IS worker, Python (activities only)
claims-stp	Comm Hub, Policy Gate, eClaims and flow activities	STP worker, Java (activities only)
claimspro-activities	Every ClaimsPro call; rate limit on the queue	ClaimsPro worker, Java

Connections and deployment are as in Approach A: outbound gRPC with mTLS through PrivateLink, Direct Connect to on-premises systems, Kong for inbound calls, one EKS deployment per queue scaled on its latency.

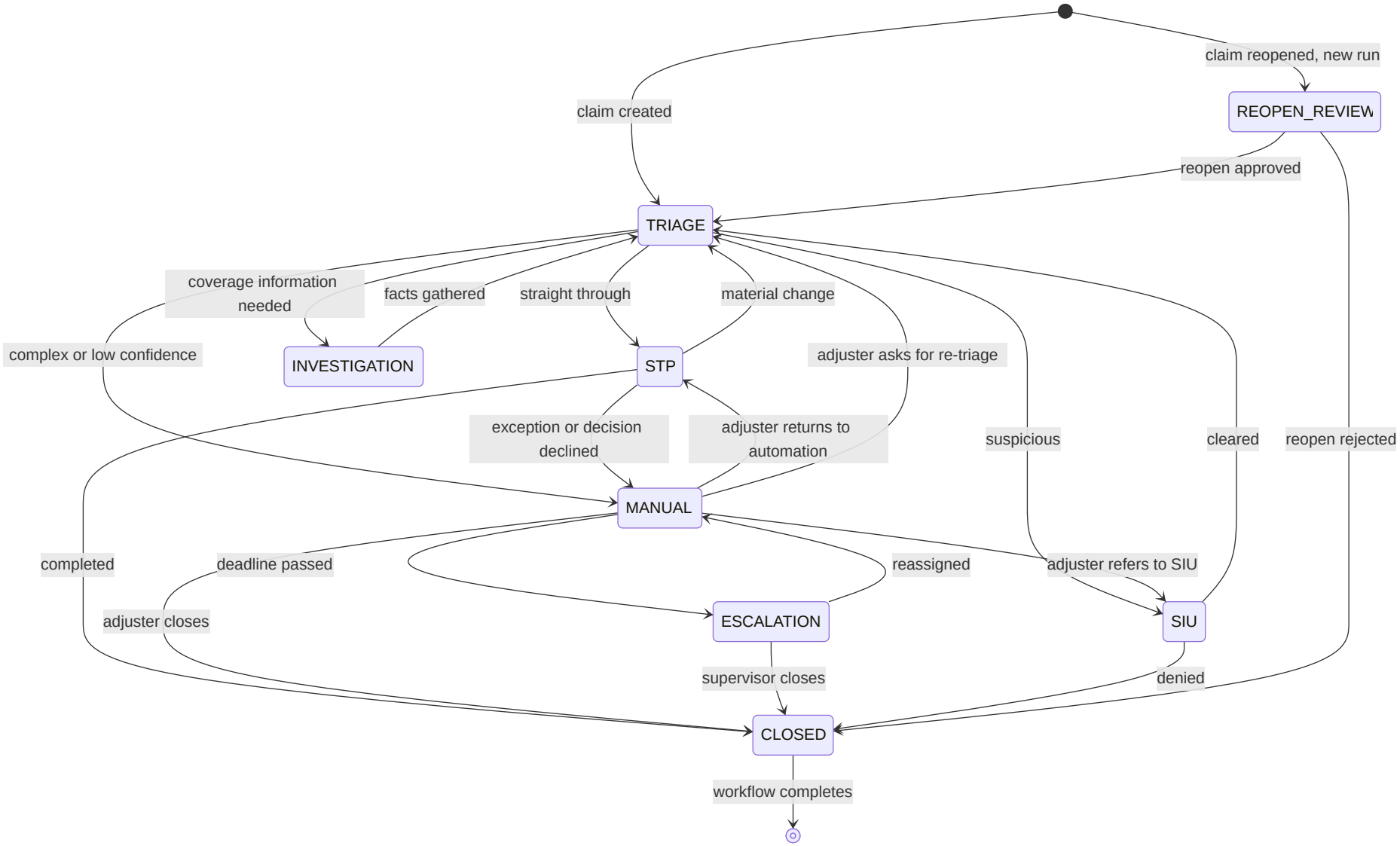
4. Inside the claim workflow



Module	Responsibility
Stage machine	The claim's stage and stage version; routing between stages; re-triage limit; Continue-As-New between stages
Triage module	Read claim, AI risk, triage rules, triage review when confidence is low; returns a route
Investigation module	AI coverage analysis; waits for documents or an IS reviewer; returns to triage

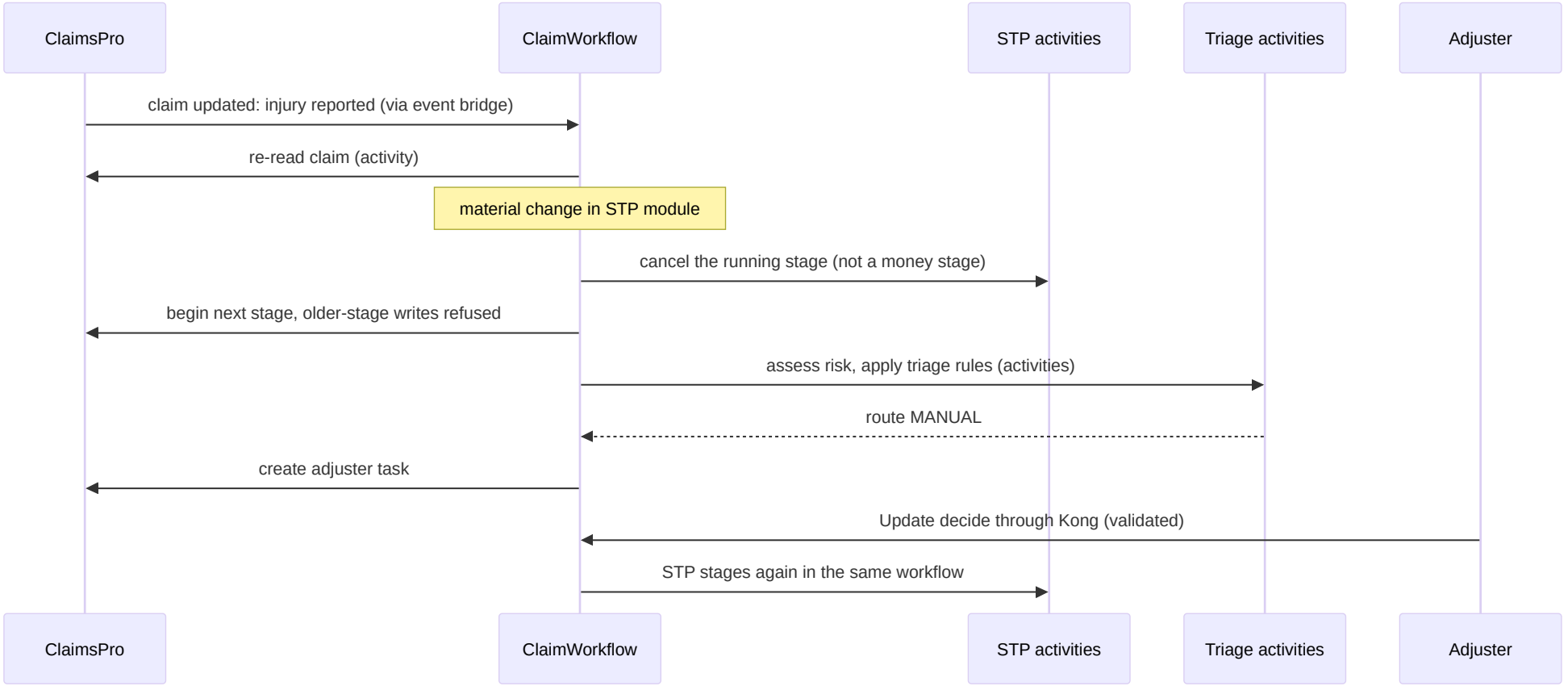
Module	Responsibility
STP module	Runs the claim's flow definition: stages in order, steps of a stage in parallel; vendor dispatch, appraisal, estimate, reviews
Payment region	Approval above authority, create (payment key looked up first), close; run outside every cancellation scope
Decision desk	Every human decision: task in ClaimsPro, deadline, escalation, validated Update

Claim stages



Every stage change increments the stage version in the projection table; writes from an older stage are refused. The workflow completes when the claim closes; a reopen later starts a new run with the same workflow ID and continues the stage numbering. It continues as new between stages when the history is large, carrying its state, so one claim's history never approaches Temporal's limits.

A claim change during STP



Events are deduplicated and handled where they are needed, in the same workflow. On a claim change the workflow re-reads ClaimsPro and applies the material-change rule. A material change cancels the running STP stage at once (its activities are cancelled) and moves the claim to triage; the payment region is never cancelled and finishes first.

5. Human decisions

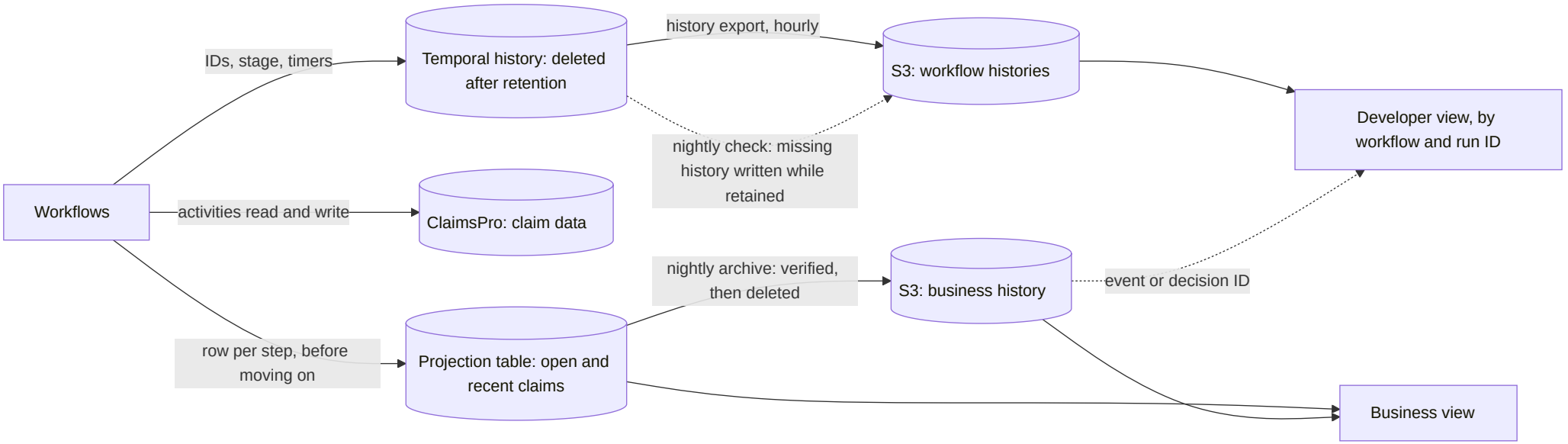
Every decision follows one lifecycle in the workflow: adjuster task created in ClaimsPro with the claim ID and decision ID; deadline timer; escalation through the roles in the authority matrix; the adjuster's decision sent by the service behind Kong as Update `decide` to `claim:{claimId}`, decision ID as Update ID; validation before anything is

recorded (task open, choice offered, role, authority, requester is not the approver, reason where required); task closed, row recorded; open tasks withdrawn when the claim moves on.

Decision	Module	Role, then escalation
Triage review (low AI confidence)	Triage	Triage adjuster, triage supervisor
Manual handling; escalation; SIU; reopen review	Stage machine	Adjuster, supervisor, SIU group
Coverage information	Investigation	IS reviewer, supervisor
Customer not reached; no vendor; estimate review	STP	Adjuster, appraiser, supervisor
Payment above authority (four eyes, reason to reject)	Payment region	Approver with authority, claims manager
A step still failing after its retry window	Any	Claims operations

6. Data outside Temporal and S3

Option for S3	Where it applies	Use here
Temporal Archival (S3 archiver)	Self-hosted Temporal only; experimental; not supported in Docker	Not available on Temporal Cloud
Temporal Cloud Workflow History Export	Every closed history to the client's S3 bucket hourly, protobuf, at least once; IAM role and KMS key	Used: the developer record
Projection archive	Nightly workflow: business rows of closed claims to S3, verified, then deleted	Used: the business record



As in Approach A: Temporal holds IDs, stage and timers; ClaimsPro keeps claim data; the projection table keeps business history, written before the workflow moves on; Temporal Cloud has no Archival, so its Workflow History Export (hourly, protobuf, at least once) is the developer record in S3, with a nightly check that copies any missing history while it is still retained; the projection table is archived nightly to S3, verified before deletion. One claim, one workflow ID: the business row and the exported history join on it directly.

7. Running in production

Area	Approach B
Versioning	One long-lived workflow type on Auto-Upgrade: every change is patched and replay-tested against recorded histories. Activity workers release independently; an activity change needs no workflow versioning
History size	All tracks in one history: Continue-As-New between stages when suggested, well below the 10,240-event warning
Throughput	Workflow tasks only on claims-lifecycle; activity queues scale per track; ClaimsPro protected by the queue rate limit; capacity sized for the catastrophe peak
Cost	No child starts and no forwarded signals: fewer actions per claim than Approach A

Area	Approach B
Failure isolation	A failing activity retries within its window, then opens an operations task; a workflow code defect affects every track of the claim, so replay tests gate every release
Security, observability	As Approach A: mTLS, PrivateLink, shared payload codec, IDs only in search attributes; ELK logs, LAMP for AI, Temporal metrics

8. Approach A and Approach B compared

	A: Parent–Child Orchestration	B: Claim Entity Workflow
Temporal pattern	Parent workflow with child workflows	Entity workflow with activities
Workflows per straight-through claim	4 (claim, triage, STP, payment)	1
Who writes orchestration code	Each track team writes its own workflow (Python TSA and IS, Java STP)	The lifecycle team (Java); track teams write activities
Events	Parent forwards to the active child	Handled in place
Human decisions	Update on the waiting workflow; task carries its workflow ID	Update always on claim:{claimId}
Versioning	Parent Auto-Upgrade, children Pinned per track	One workflow Auto-Upgrade with patching; activity workers free
History	Split across child executions	One history per claim, more Continue-As-New
Actions per claim	Higher	Lower
Measured surge (200 claims, 20 per second, local)	All closed in 82 s	All closed in 58 s
Blast radius of a workflow bug	One track	The whole claim workflow
Choose when	Track teams must own and release their workflows independently	One team owns the claim process; simplicity, cost and one view of the claim matter most

9. Pros and cons

Pros	Cons and how they are handled
Simplest model: one claim, one workflow, one ID for everything	One workflow type carries all tracks: kept in modules, reviewed and replay-tested as one
No event forwarding or decision relay; nothing to keep in sync between workflows	Python teams write activities, not workflows: their AI logic is the activity anyway
Fewest actions and executions per claim	History grows with every track: Continue-As-New between stages
Activity workers deploy freely	Every workflow change must be patched, as the claim workflow lives for the whole claim
Payment and re-triage rules in one place	A workflow defect stops the whole claim, not one track: replay tests and ramped rollout

10. Answers to the questions in the design document and the kickoff call

Question	Answer in this approach
One parent coordinating TSA, IS, STP children?	Not needed. The tracks need their own workers, which activities on their own queues give; one claim workflow owns stage, routing and cross-track rules, and each track is a module of it.
New child per visit or reuse? Stage versions and lease tokens?	No children. Each visit to a track is a new stage version; every write carries it and is refused if the claim has moved on.
Complete at close, or keep a reopen window?	Complete at close, after any payment in progress finishes. A reopen starts a new run with the same workflow ID; a reopen that arrives while the run is closing is handled in that run.
Workflow ID reuse, retention, Signal-With-Start; when to Continue-As-New	Only created and reopened events start a run. Continue-As-New between stages, with no decision or payment open, when Temporal suggests it or above 5,000 events.
Projection table for years? Wait for each write? Recovery after history expires?	Yes. Awaited, idempotent rows keyed claimId:stageVersion:step; the record is verified complete before the workflow completes; histories reach S3 by export and the nightly check.
Bridge to parent to child?	Bridge to the claim workflow, which handles each event where it is used. No forwarding, no buffering between workflows.
100 to 150 events per claim	Signal per event, deduplicated by event ID; no forwarding cost.
Human approval: Update or Signal? Parent or child?	Update with a validator, always on claim:{claimId}; the service behind Kong needs only the claim ID.

Question	Answer in this approach
JSON Schema or Protocol Buffers? Compatibility?	Protocol Buffers for Java and Python. Fields only added; unknown fields and enum values read by both languages; golden payloads tested in both.
Condition change during an in-flight activity	A material change cancels the running STP stage at once (its activities are cancelled); payment and close finish first; the stage version refuses any late write.
YAML/JSON flow definition	STP flows from the variations tool: registered steps, stages run in order, steps of a stage in parallel; version and hash recorded. Payment authority has a ceiling in code.
Auto-Upgrade parent, Pinned children?	One workflow type on Auto-Upgrade with patching and replay tests; activity workers release without workflow versioning.
Pass references or data? Rerun must not pay twice; cancel the payment	References. Payment key looked up before create (verified with a lost reply). A cancel from ClaimsPro stops the payment region until it closes.
Business view across open and archived claims; developer view months later	Projection table and S3 archive share one schema; one workflow ID per claim opens its whole exported history.
Pilot scope: Claims Lifecycle to STP only	A policy switch skips triage and investigation; a re-triage goes to an adjuster.
Catastrophe load	Measured locally: 200 claims at 20 per second, all closed in 58 seconds with one payment each and complete records, with failures injected.
PrivateLink, encryption	Outbound only over PrivateLink with mTLS; payloads and failure messages encrypted with one codec in both languages.

11. Claim scenarios verified end to end

Scenario	What it proves
Straight through	Full STP; a lost reply from ClaimsPro still gives exactly one payment
Injury during STP	The running estimate stage is cancelled at once; re-triage; an adjuster returns the claim to STP
Coverage investigation	Investigation waits; documents arrive; re-triage; STP
Fraud	SIU denial without a reason is rejected; with a reason it is accepted

Scenario	What it proves
Large payment	Estimate review; approval rejects the requester and an approver without authority
Catastrophe, lost event	Coverage, reserve and appraisal in parallel; the lost estimate event is recovered by the timer re-read
Customer not reached	An adjuster asks for another contact attempt
Low AI confidence	A triage adjuster chooses the route
Closed in ClaimsPro, reopened	The running module is cancelled and its task withdrawn; the reopen is a new run
Payment cancelled in ClaimsPro	A cancel during approval stops the payment; an adjuster closes the claim with a reason